Assumptions that were never used

Carlo Perassi

September 2026

Abstract

We describe a mechanical search for theorems in a large formal library whose stated algebraic setting is stronger than the setting their own proof requires. The question is not new: it was answered analytically for this library in 2021, and the corresponding brute-force method — weaken the hypothesis, re-run the proof — was carried out on a different library in 2013 and judged, in the same sentence that cites it, too slow for this one. We report what it costs, and we run it over two libraries rather than one.

We examined a *sample* of 29,261 theorems of Mathlib, across 18 subject areas, and found 431 that hold with a weaker hypothesis than the one they are stated with: 1.47%, about one in 67, spread over 240 files. The sample is not random — it is what the search could reach — and re-weighting it to the library's own shape gives 1.71%, so the composition costs little and costs it in our favour. The same instrument on Tau Ceti [6] — a corpus of formal mathematics written by AI contributors rather than by people — finds them at 0.45% of attempts against 0.40% here, intervals [0.36, 0.57] and [0.36, 0.44], $p = 0.35$. We can measure no difference at a resolution that would have shown one of half again as much. Whatever this is, it is not a habit of one library's authors.

Per declaration the method remains some fifty times more expensive than reading the proof term, so it is not the interactive tool that judgement was about; but a library fits in a few days of one machine, which is a bound nobody had put on it. None of the survivors is new mathematics, and in 87.5% of the rows we could check the weakened proof is the same proof term, so the assumption was genuinely unused rather than merely replaceable by a search. What they give up is invertibility — subtraction or division — and in Mathlib they do so far more often in areas built on invertible structure than in areas without it. The phenomenon is not spread evenly across subjects either: one rate does not explain the areas we swept, the extremes differ by a factor of 8, and we do not know why.

1 What we did and what came out

Take a theorem from Mathlib. Replace one instance binder with a weaker class. Keep the proof byte for byte. Compile the result alone, and keep it only if Lean reports no error and `#print axioms` shows nothing beyond `propext`, `Classical.choice` and `Quot.sound`. No judgement enters, and nothing survives that the compiler has not re-checked.

Three things came out, and they are the whole of what we claim.

One. It is affordable. A pass over the library is about four days on one laptop, covering 7,869 of 8,311 source files, 95%. That contradicts nothing anyone measured, and settles something nobody had: the method is a batch instrument, not an editor-time one, and its batch cost is bounded.

Two. What gets given up is invertibility, and where depends on the area. Of the theorems we can weaken in the four areas we swept first, 44 — 13.7% — remove the ability to subtract or divide. In areas built on invertible structure the rate is 51% ($p = 3.1 \times 10^{-8}$); in areas with nothing to invert it is 4.5%. We pre-registered that contrast and tested it on seven areas that played no part in forming it: 25% against 0%, $p = 0.016$.

Three. The phenomenon is not spread evenly. Across 21 completed areas the rate runs from nothing to 29.9 per thousand declarations. One rate does not explain them: $G = 49$ on 20 degrees of freedom, $p = 0.0003$, clustering by file.

One theorem can be weakened in more than one way, so 431 theorems give 462 distinct weakenings; the method counts attempts rather than theorems, and Section 2 says how the two relate. A fourth, which we set out to test rather than to find, is the 87.5% in the abstract: the weakened proof is usually the same proof term. Sections 3 to 7 take these in turn. Section 11 says what none of it shows, which is more than usual. Four worked examples are in Appendix A; all 462 are in the companion catalogue, unfiltered, because no filter we have can judge them.

2 Method

The substitution. A weakening table maps each class to classes weaker than it. We use two, and they are not nested in either direction. One is hand-written, 48 edges over the hierarchy as we read it. The other is derived from the library's own `class ... extends` declarations, 374 edges over 224 classes. Over the fifteen areas swept both ways the hand table finds 460, the derived 379, with 199 in common and 640 in the union — 39% more than either alone. A derived table cannot propose a *sibling* edge, and some of the most productive edges are between siblings; a hand table cannot name classes nobody thought to list. Every count here is therefore a lower bound, and the factor is measured rather than guessed.

Three filters before any compile. Most proposed weakenings are dead on arrival, and rejecting them cheaply is what makes the pass affordable. 58% of attempts are settled without invoking the compiler: the binder named is not one the theorem carries, or the assumption never reaches the conclusion, or the substitution is not a weakening. One `#check` per declaration decides these.

A prior-art gate. Every nomination is put to `exact?` on the weakened statement, and dropped unless nothing usable comes back. A row where the library's own search closes the goal is not our finding. A row whose citation is the theorem's own name means the binder was never a hypothesis.

Theorems, weakenings, attempts. Three units. The paper uses whichever one the sentence is about, and they differ by more than rounding, so the table says which is which.

unit	what it counts	Mathlib
theorem examined	a declaration for which at least one weakening was proposed and decided	29,261
theorem weakenable	of those, one that holds with a weaker hypothesis than it is stated with	431
weakening	one binder of one theorem holding over one weaker class; a theorem may have several	462
attempt	one candidate class put to the compiler; a theorem generates many	116,265

The two corpora, on both denominators:

	Mathlib	Tau Ceti
theorems in the corpus	186,856	29,589
examined	29,261	5,526
as % of the corpus	15.7%	18.7%
weakenable	431	62
as % of examined	1.47%	1.12%
as % of the corpus	0.23%	0.21%

About one theorem in six is examined at all; the rest carry no instance binder either table can weaken. So 1.47% is the rate among theorems the method can ask about and 0.23% is the rate in the library; Section 11 says why neither alone is the honest number. Both corpora agree on both denominators, which is a stronger statement than the yield comparison of Section 6 alone: the rates there are over attempts, and these are over theorems. Attempts are reported for cost, not for yield.

Every survivor is a *weakening*: a class replaced by a weaker one. No binder is ever deleted, so nothing here counts an assumption removed outright.

Which set each number counts. Three populations appear below and are not interchangeable, so each is named where it is used. The *discovery set* is Algebra, Analysis, NumberTheory, RingTheory: 322 survivors from 73,620 attempts, where the pattern was found, and every statistic about the pattern is computed over it. The *full hand-table sweep* is 18 areas, 462 survivors from 116,265 attempts, and contains the discovery set. The *derived-table sweep* is a second pass with the other table, reported separately throughout; Section 7 is computed over it and says so.

3 The cost

Best [1] inspects a theorem's elaborated proof term and reports which typeclass fields the proof never touches. In the same paper he sets our variant aside, and we quote him entire because the elision matters:

> a brute force strategy of weakening typeclasses in theorem statements and re-running the same proof script seems far too slow to be used on the scale of a large library in a prover such as Lean, however similar techniques have been used in the Mizar system [12]. Nevertheless, a tool to automate this process could still be useful for small new developments.

His [12] is Alama [2], who did exactly this to the Mizar Mathematical Library a decade before. On the reading Best meant — a tool usable while formalising — he was right, and our own numbers confirm it: term inspection handles a 475-declaration file in one to two minutes, and substituting and recompiling the same file costs us something over an hour. Fifty times more expensive per declaration, and no hardware since closes that.

What we can add is the bound he did not state. At 18 attempts per file and 26 decisions a minute, a complete pass is a few days of one machine. Not a cluster, not a grant. The derived table has now been over 25 areas, 7,869 files, 95% of the library; the 442 files left are Tactic, Lean, Util, InformationTheory, Testing, Deprecated, which hold no theorems of the kind this looks for. The extrapolation is no longer an extrapolation.

4 Invertibility

Of the 322 survivors in the four discovery areas (Algebra, Analysis, NumberTheory, RingTheory), 44 — 13.7% — give up subtraction or division. That is the baseline. The 462 of the abstract is the full hand-table sweep, 18 areas; every statistic about the pattern is computed over the 322, because that is where the pattern was found.

The claim is about where. An area whose objects carry inverses — groups, rings, fields, modules over them — should yield survivors that surrender them; an area with no inverses to surrender cannot. We named three areas and predicted each before sweeping it:

LinearAlgebra	10/22	45%
Combinatorics	2/8	25%
GroupTheory	8/9	89%

Against 13.7% at baseline and 4.5% in areas where these weak structures are already the subject, the three together give 20 of 39 — 51%, $p = 3.1 \times 10^{-8}$.

One of the three predictions was wrong. We expected Combinatorics low, reasoning that counting arguments run over the naturals where subtraction is unavailable. It sits high, and the explanation — that this part of the library is largely additive combinatorics, conducted in groups — is one we can give only after seeing the number. We report it because a scale reported only where it worked is selection rather than evidence.

The statistic was chosen after seeing the data, so we re-ran it as a pre-registration. The hypothesis, the classification of each area, the test and the significance level were written down and committed before the measurement. Seven areas that had played no part in forming the claim: those built on invertible structure gave 4 of 16, those without gave 0 of 26. Fisher exact, two-sided, $p = 0.016$. The gradient is much shallower than the post-hoc figure — 25% against 51% — and what carries it is the low end, 0 of 26 in areas with nothing to invert. Read the 51% as what post-hoc selection does to a real but modest effect.

5 Never used, or merely not needed?

A row says the theorem compiles with a weaker hypothesis. That covers two facts. Either the elaborated proof term is the same one and the assumption was dead weight, or the tactic, re-run in the weaker setting, searched again and found another route — in which case the assumption was not unused but *replaceable*. Best predicted the second and did

not measure it, expecting it "especially likely with small proofs that make heavy use of automation".

We measure it by elaborating each weakened declaration alongside the original, reading both proof terms out of the environment, and comparing the constants each cites. Typeclass plumbing is excluded — substituting a class necessarily changes which classes, parent projections and instances a term mentions, so counting those would call every row replaceable. What remains is the lemmas the proof appeals to.

Of 618 rows we could classify, 541 — 87.5% — cite exactly the same lemmas: the same proof, and the assumption was never used. 77 (12.5%) cite different ones. 27 could not be classified at all, because the original declaration is `private` and cannot be named from outside its file; those rows verify like the rest and are reported separately rather than folded in.

A companion project [8] measured the same question at a different strength, reading only the *root* of each proof term — the lemma the proof ends on — and found it unchanged in 99.15% of comparable rows against our 87.5% over the whole cited set. The two are nested, and joining them row by row leaves the impossible cell empty: no row has a moved root and an unchanged citation set. That empty cell is the only evidence either project has that the two measurements measure what they claim; a single row in it would have convicted one pipeline without saying which. The gap between the two is the quantity neither can state alone. Roughly one weakening in eight keeps its destination and changes its route.

Two things this did not show, both predicted. The gradient of Section 4 is unchanged when restricted to the genuinely unused rows — 17.4% of them give up invertibility against 17.4% of all rows, $p = 0.63$ — so nothing in that argument rests on the distinction. And replaceability is not concentrated in proofs that visibly search: 14% of rows using `simp` or a similar tactic are replaceable against 12% of those using none, $p = 0.36$. Whatever makes a proof replaceable, it is not simply that a tactic went looking.

6 The same rate in a library no human wrote

If the pattern were a habit of Mathlib's authors, a corpus written by a machine should not share it. Tau Ceti [6] is such a corpus — formal mathematics directed by human-written roadmaps, implemented by AI contributors and subject to adversarial review, incubated by the Lean FRO and the Mathlib Initiative. Swept with the same hand-written table — 42 of its 48 edges appear in that store and none outside it — it yields 70 survivors from 15,602 attempts, 0.45%, against 0.40% over the full hand-table sweep here. Intervals [0.36, 0.57] and [0.36, 0.44]; Fisher exact, two-sided, $p = 0.35$; ratio 1.13. They overlap.

Say what that can and cannot exclude. At 15,602 attempts against 116,265 the comparison has power 0.96 against a corpus twice as over-constrained and 0.78 against one half again as much. A difference of 50% would probably have shown. This is a null result with a stated resolution, not a demonstration that the rates are equal — and the resolution is the point: an earlier pass on a 4,000-attempt sample of the same corpus could only exclude a doubling.

The comparison is of *yield*. Whether the invertibility gradient of Section 4 also survives a change of author is a separate question and we cannot answer it here: Tau Ceti's 25 gated survivors sit almost entirely in areas built on invertible structure, so there is no low group to compare against. Its survivors give up invertibility at 6 of 25 against 44 of 322 here, $p = 0.23$, which is consistent and settles nothing.

No prior work makes this comparison. Best ran a single pass over one library and says

so; the question of whether unused assumptions are an artefact of human authorship needs a corpus no human wrote, and one now exists.

Attempts is the wrong denominator for a comparison between libraries and we use it because it is the only one available: a table with more edges per class produces more attempts per declaration and so a lower rate at identical yield. The defensible sentence is that we can measure no difference, not that the rates are equal.

7 Distribution

Counting survivors rewards an area for being large. The question underneath is whether a theorem in one area is *more likely* to carry an unused assumption, which needs a denominator: declarations for which some weakening was proposed. The numerator is theorems with at least one survivor, not survivors, since one theorem can yield three and those are not three independent trials.

This section alone is computed over the derived-table sweep, which reached the end of the library. Across 21 completed areas the rate runs from nothing to 29.9 per thousand declarations, pooled at 9.8. Taking theorems as independent trials gives $G = 126$, $p = 2.7e-17$ — and they are not independent trials. Mathlib heads a file with `variable [Field K]` and every theorem below inherits it, so the assumption being weakened belongs to the file at least as much as to the theorem. At the observed design effect of 2.6 the statistic falls to $G = 49$, $p = 0.0003$, which still rejects one rate. Under the hand table the same correction leaves no significant result, $p = 0.13$, and we report that rather than only the instrument that agrees with us.

Two objections, both answerable from the same data. A dense area might simply have had more attempts per declaration, but the rank correlation between rate and attempts per declaration is -0.23 ($p = 0.31$). And the ranking might be a picture of our weakening table rather than of the library; re-measured under the other table the levels move a great deal and the order holds at $\rho = +0.80$ over 20 areas ($p = 2e-05$). The level is the instrument's. The order largely is not.

Two things this is not. The two tables are not independent, both being orderings of the same hierarchy, so that agreement is agreement rather than replication. And the denominator counts only declarations some table could propose a weakening for, so this is density among the theorems the method can ask about, which is not density among theorems. We do not know what the second would be.

8 Who uses them

A weakening that holds is not thereby wanted. Naming conventions speak to a library's habits; citations speak to its use.

A companion project [8] counted, over the whole library at our revision, how many declarations cite each survivor — from proof terms, not from names. Of the 571 survivors it could join, 174 are never cited at all: 30.5%, interval [26.3, 35.3] after correcting for clustering by module at a design effect of 1.42. Against non-survivors *in the survivors' own modules* — 20,125 theorems, same files, same authors, same density of machinery — 37.7% are never cited. The gap is 7.2 points, survivors lower, $p = 0.0018$.

Wider comparisons give a wider gap, and the difference between them says why: 15.7 points against every other theorem in the corpus, and 12.4 after dropping the 22,854 declarations Mathlib generates rather than writes (`.injEq`, `.noConfusion`, match auxiliaries), which are largely never cited by name. About half the raw difference is machinery nobody

cites and nobody wrote. The same-module control is the honest comparison and is the one we quote.

So a redundant hypothesis is not a marker of dead code, and not a marker of infrastructure either. It sits in theorems used a handful of times each and then not examined again: mean 2.34 against 4.28 for their neighbours, and the most-cited survivor is cited 193 times against 4,249 for the most-cited neighbour. A lemma a thousand proofs depend on has been read by many people; a lemma nothing depends on was perhaps never finished.

Mathlib's own conventions say the same thing more weakly. Of the 293 survivor declarations our locator could find in the source, 47 carry a docstring at all (16%), and 1 has a docstring naming a classical result in bold, which is how Mathlib marks one: the *Shrinking lemma*, weakened from a metric space to a pseudometric one. A handful of others are recognisable — `riesz_lemma`, the Hadamard three-lines bounds — but by surname in the declaration name only 22 of 462 match, and most of those are namespace coincidences rather than named theorems. Those tests are about naming rather than use, and our locator failed on a quarter of the rows. The citation counts are the measurement and they agree.

9 What can actually be applied

A row is not a patch. 89% of the assumptions we weaken are written on a `variable` line, so they belong to a section rather than to a theorem: weakening one there changes every declaration below it, and most of those still need the stronger class. A patch set derived row by row would be wrong more often than right.

There is a subset where it is safe, and we let the compiler decide which. 102 groups have the property that every theorem carrying the binder also survived — the property one would call safe on the evidence. Of the 68 where the binder is actually on a `variable` line, we rewrote it, rebuilt the whole file against the pinned revision, and kept what compiled: **36**. The compiler rejected 32, very nearly half.

That rejection rate is the useful number. A declaration the search never tried can still need the stronger class, and no amount of reasoning over our own verdicts finds it — only building the file does. It is the same lesson as the descent floor: a stage that computes something must keep what proved it.

The 36 that survive are one-line changes against a named revision, each with the file it belongs to, shipped in `patches/` with every rejection and its first compiler error beside them. They are the smallest actionable thing this work produces, and the only part of it a maintainer can evaluate without reading the rest.

10 Relation to prior work

The question has been asked of this library and answered analytically. Best [1] produced 2,877 suggested generalisations in a single pass by reading elaborated proof terms. The substitute-and-recompile method is older and belongs to Alama [2] on the Mizar Mathematical Library; Pons [9] is the earlier root of the analytic branch.

Neither method contains the other, and Best's published Table 1 settles it in both directions. His table lists fourteen assumptions against some fifty replacements, so a comparison needs a rule; ours is every (original, replacement) pair he records ten or more times, which is 25 of them. 13 have no counterpart in our substitution table at all: they relax an assumption to a bare operation — an order to its $<$, a module to its scalar action — and a table of named structures cannot name a target that is a fragment of a class.

Conversely, a pass that reads one proof term never re-elaborates the weakened statement, and 75% of our attempts fail precisely there: a sibling assumption demands the class back, nothing typechecks, and there is no term to inspect. Where both look at the same edge they agree closely — field to division ring is 23 for him and 25 for us. The per-theorem overlap stays unmeasured, because his metaprogram targets the previous prover and the previous library.

Gandhi, Tadipatri and Gowers [3] give a Lean tactic that generalises the *constants* in a proof, and set our target aside in one sentence: "it is unlikely that any proof-generalization algorithm would be useful when applied to results in Lean's mathlib, since proofs there are already designed to be as general as possible." We are careful about what our data does to that. It does not show a generalisation algorithm is *useful* on mathlib; usefulness needs somebody to want the output, and nothing here has been offered upstream. What it bears on is the stated reason. Proofs there are not quite already as general as possible, at a measured rate of 0.44% of attempts — small, and not zero.

Baanen [4] is the standing account of instance parameters in this library, which is the object we manipulate. Mathlib's own maintenance report [5] describes an `unusedVariable` linter that already catches hypotheses which turn out to be superfluous; anything a shipped linter finds is not ours to report, which is one more reason the prior-art gate runs before anything is counted.

11 What this does not show

No novelty is claimed. A reader will grant each survivor on sight. The library's own search cannot prove them, which is a much weaker statement than novelty, and it is the only one we make.

Nobody has said they want these. None has been offered upstream. The difference between a weakening that is true and one that is wanted is the whole value of the result, and we have evidence for only the first. What we can offer is 36 one-line patches that a maintainer can apply and check in a minute each (Section 9), and the evidence of Section 8 that these theorems are used rather than dead.

Nothing is claimed about how far the gap goes. Each row records the weakest class it is *verified* to prove, and a search below that reached lower classes we could not recompile; the published data carries those in a column marked unverified, and no statistic here uses it. The ordinal measuring the distance is also undefined for a pair of classes neither table places, so an area with more unplaced classes looks shallower whether or not it is — 8 of Order's 44 survivors against 22 of the other 299. Depth is reported per row and not aggregated. The invertibility results do not use it.

The pipeline has shipped defects, and they are documented. Every one had the same shape: something that resolved nothing looked exactly like something that contained nothing, and both print zero. The artifact [7] carries the data, the tooling, a verifier that recompiles every published row from a clean checkout, an `ENGINEERING.md` giving each defect with its diagnosis, and one regression test per defect in `tests/`.

The examined set is a sample, and not a random one. Areas were chosen rather than drawn, and within an area a theorem enters only if the collector can locate and rewrite one of its binders. The composition is therefore skewed: RingTheory is 2.2×

over-represented against its share of the library and Order 0.7× under-represented. What matters is whether the skew moves the answer. Re-weighting each area's own rate by its share of the corpus rather than by its share of what we examined gives 1.71% across the 15 areas with at least 200 theorems examined, against 1.47% as measured — a factor of 1.07, and upward. The measured rate is if anything conservative, but every figure here should be read as a sample statistic and not a census.

The rate over the corpus is a floor, not an estimate. 0.23% divides 431 by every declaration in the library, including those the search never looked at, and it is reported that way because the alternative is to quietly choose a smaller denominator. The library divides into three groups: 16% of declarations carry no instance binder in scope, so there is nothing to weaken and they are out of scope by definition; 17% carry binders whose classes neither of our tables covers, which is a limit of the tables; and 67% carry a class we could weaken. We examined 29,261, well under that 67%, because the hand-table sweep ran 18 areas rather than the whole library and because of the binder-locating limit above.

So 1.47% is a measurement over the theorems the method examined, and 0.23% is what that implies for the library only if the unexamined behave like the examined. We have not tested that, and the second number should be read as a lower bound rather than a rate.

One library, one revision. Every row is relative to a named Mathlib commit. The artifact publishes 645 rows — the union of both weakening tables, so more than the 462 this paper computes over — and 644 of them recompile from a clean checkout at that revision. The one that does not is published too, with its error, because a dropped row is a claim nobody can check.

12 Open questions

We are not expert in these areas and offer the following as observations rather than claims.

- Observation 1 is an ultrametric bound over a commutative ring where the library states it over a field. Our descent stage reached a semiring, which would be the more interesting object, but we could not reproduce that compile and do not publish it. Is the bound over a commutative ring a studied object, and does it in fact go down to a semiring?
- Observation 2 admits exactly the extended non-negative structures. Are there results about $\mathbb{R}_{\geq 0} \cup \{\infty\}$ currently stated over groups, so that the group is a wrapper around an argument that never used it?
- Our hand table names no categorical class. Swept with the derived table, CategoryTheory gave 1 survivor from 703 attempts, and 28% of attempts could not be reconstructed at all: a categorical hypothesis is usually a relation between objects, and this method replaces one assumption on one variable, so the question is less unanswered than unaskable by this instrument.

A Four worked examples

Chosen by reading, not by any filter. All 462 survivors are in the companion catalogue, grouped by area and weakening, each with the signature Lean prints for it.

Observation 1 (An ultrametric bound needs no field)**.** *Let R be a commutative ring and $v\colon R \to \mathbb{R}$ an absolute value with $v(x+y) \le \max\{v(x), v(y)\}$. For finite I, J, $A = (a_{ji}) \in R^{J\times I}$ and $x \in R^I$,*

$$\sup_j v\Big(\sum_i a_{ji}x_i\Big) \;\le\; \Big(\sup_{j,i} v(a_{ji})\Big) \cdot \sup_i v(x_i),$$

and with a factor $|I|$ on the right, the same holds without the ultrametric hypothesis. Both are stated in the library over a field.

Observation 2 (Comparison after collapsing a top element)**.** *Let α carry a partial order and a commutative associative addition with a zero — no subtraction — and let $u\colon \alpha\cup\{\top\} \to \alpha$ fix α and send $\top$ to 0. Then u is monotone away from the top, reflects order under the expected side condition, and is non-negative when α is. Stated over an ordered abelian group.*

Observation 3 (A rational floor certificate needs no inverses)**.** *If a rational certificate for an element of an ordered ring already carries the invertibility datum it needs, the resulting statement about its floor holds over the ring. It is stated over a field, and a decision procedure in the library constructs the field instance for no other purpose than to apply it.*

Observation 4 (Sup and inf are uniformly continuous without division)**.** *Let α carry a linear order and a ring structure whose order is strict, and nothing further — in particular no inverses. For ε and a_1, a_2, b_1, b_2 in α, if $|a_1 - b_1| < \varepsilon$ and $|a_2 - b_2| < \varepsilon$ then $|a_1 \sqcup a_2 - b_1 \sqcup b_2| < \varepsilon$, and the same with $\sqcap$. Both are stated in the library over a field. Each proof is one line, and cites a lattice inequality and transitivity.*

References

[1] Alex J. Best. *Automatically Generalizing Theorems Using Typeclasses*. Workshop on Formal Mathematics for Mathematicians, CICM 2021. EasyChair preprint 6216.

[2] Jesse Alama. *Eliciting Implicit Assumptions of Mizar Proofs by Property Omission*. Journal of Automated Reasoning 50(2):123–133, 2013.

[3] Anshula Gandhi, Anand Rao Tadipatri and Timothy Gowers. *Automatically Generalizing Proofs and Statements*. ITP 2025, LIPIcs vol. 352, article 12. `doi:10.4230/LIPIcs.ITP.2025.12`.

[4] Anne Baanen. *Use and Abuse of Instance Parameters in the Lean Mathematical Library*. ITP 2022, LIPIcs vol. 237, article 4.

[5] Anne Baanen et al. *Growing Mathlib: maintenance of a large scale mathematical library*. arXiv:2508.21593, 2025.

[6] The Tau Ceti Project. *Tau Ceti: a repository of formal mathematics implemented by AI contributors*. Swept at revision `675a2d2c`. `https://github.com/TauCetiProject/TauCeti`

[7] Data, tooling and a verifier for every row in this note. `https://github.com/carlok/unused-assumptions`

[8] The `unstated-conclusions` project, 2026. Proof-term roots and citation counts over the rows of this note; both are published row by row in [7].

[9] Olivier Pons. *Generalization in Type Theory Based Proof Assistants*. Types for Proofs and Programs, LNCS, 2002.